Table of Contents

SECTION 01

Executive Summary

This document presents a comprehensive build plan for a professional web portfolio deployed on GitHub Pages, designed to position the candidate across AI/ML, Data Science, and Full-Stack engineering roles. The portfolio blends Modern Bold typography with Dark Sleek aesthetics, anchored by a Dark Neon Blue base color with Neon Silver/Platinum accents. Every section, layout decision, and technology choice below has been derived from current industry standards for developer portfolios in 2026, with particular attention to what hiring managers and technical recruiters actually look for when evaluating candidates in AI/ML and adjacent fields.

The plan covers positioning strategy, technology stack selection, a complete design system specification, section-by-section content architecture, AI/ML-specific showcase elements, feature checklist, deployment workflow, and a phased build execution order. It is structured so that the candidate can verify each layer before any code is written, and so that downstream implementation can proceed in clean phases with measurable deliverables at each step.

TARGET ROLE

AI / ML

Primary positioning, with Data Science as secondary and Full-Stack as tertiary credibility layer.

TECH STACK

Next.js 16

App Router + TypeScript + Tailwind CSS 4 + shadcn/ui + Framer Motion, statically exported to GitHub Pages.

DESIGN THEME

Neon Blue

Dark Neon Blue (#00d4ff) with Platinum Silver (#c8d0e0) accents, blending Modern Bold and Dark Sleek aesthetics.

DOMAIN

github.io

Free GitHub Pages subdomain (username.github.io), with custom domain option preserved for future migration.

The remainder of this document walks through each layer of the plan in detail. After review, the candidate should approve the plan as-is or request changes before the build phase begins. Three material inputs are required before implementation: the candidate's resume (PDF or text), LinkedIn profile URL, and GitHub profile URL. A short list of optional but high-impact inputs (target companies, project screenshots, headshot, email address, analytics preference) is also collected at the end of the document.

SECTION 02

Research Findings: Modern Portfolio Standards

Before drafting the plan, current industry standards for developer portfolios were researched across multiple sources covering general design trends, AI/ML-specific expectations, Data Science portfolio practices, Full-Stack conventions, GitHub Pages deployment patterns, and recruiter-side evaluation criteria. The findings below represent the consensus view across these sources as of late 2025 and early 2026, and they directly inform every architectural and design choice in this plan.

2.1 Cross-Discipline Consensus

The research surfaced a clear consensus across all four target disciplines (AI/ML, Data Science, Full-Stack, Web Dev) on what a modern portfolio must contain and how it should be built. The table below summarizes the seven dimensions that appeared repeatedly in best-practice guides, recruiter interviews, and award-winning portfolio case studies. These dimensions form the baseline against which this plan is calibrated.

Dimension	Industry Standard (2026)
Tech Stack	Next.js 16 / Astro + TypeScript + Tailwind CSS, statically exported to GitHub Pages. The React-based Next.js stack remains the dominant choice for developer portfolios in 2026.
Must-Have Sections	Hero, About, Skills, Experience, Featured Projects, Resume download, Contact. Universally cited across recruiter and developer sources as the baseline minimum.
Recruiter Priorities	Real impact with quantified metrics, tech-stack match, evidence of problem-solving ability, clean and readable GitHub code, and an easy-to-find contact path.
AI/ML Specifics	Models with measurable metrics (accuracy, F1, latency, throughput), datasets, experiment tracking (W&B; / MLflow), paper implementations, and live demos or notebook links.
Data Science Specifics	End-to-end project narratives: business problem, data acquisition, methodology, visualization, and measurable business impact. Reproducible notebooks are expected.
Full-Stack Specifics	Architecture diagrams, live deployed demos, both frontend and backend evidence, CI/CD pipeline visibility, and explicit deployment target documentation.
Design Trends	Dark-mode-first with toggle, bold typography (Space Grotesk, Inter), smooth scroll animations (Framer Motion), restrained glassmorphism accents, micro-interactions.
Performance	Lighthouse score 90+, LCP under 2.5 seconds, mobile-first responsive layout, static generation for fast first paint, lazy-loaded media assets.

2.2 Recruiter Perspective

Recruiter-side sources were particularly valuable in calibrating the plan. The consistent message from hiring managers and technical recruiters was that portfolios are still actively reviewed, especially at later interview stages and for senior or specialized roles. AI-assisted resume screening has become common, but human reviewers continue to use portfolios as a differentiator between candidates with similar resumes. The three primary filters recruiters apply are work history, tech-stack match, and educational background, but a strong portfolio can override weaknesses in any of the three by providing direct evidence of capability that a resume cannot convey.

Specifically, recruiters look for outcomes and metrics rather than lists of technologies. A project described as "fine-tuned BERT classifier achieving 0.94 F1 on a 50k-row imbalanced dataset, deployed behind a FastAPI endpoint serving 200 requests per second" is dramatically more compelling than "built a sentiment analysis model using BERT." This insight directly informs the Featured Projects architecture in Section 6, which mandates a metric field for every project card.

SECTION 03

Positioning Strategy

The portfolio positions the candidate across three layers of professional identity, with explicit primary, secondary, and tertiary roles. This layered approach lets the portfolio speak to multiple job families without diluting the message: a recruiter scanning for ten seconds sees a clear primary positioning, while a hiring manager digging deeper discovers the supporting breadth. The hero tagline condenses this layering into a single line: "AI/ML Engineer · Data Scientist · Full-Stack Developer."

3.1 Role Layering

Layer	Role	How It Shows Up
Primary	AI / ML Engineer	Lead with models, experiments, datasets, paper implementations, metrics (accuracy, F1, latency), and live demos. This is the dominant lens through which the portfolio reads.
Secondary	Data Scientist	End-to-end project stories that walk through business problem, data acquisition, methodology, visualization, and measurable impact. Reinforces analytical depth.
Tertiary	Full-Stack / Web Dev	Evidence that the candidate can build and deploy the systems that serve ML models in production. Establishes generalist credibility and removes the "research-only" stereotype.

3.2 Why Layered Positioning Works

A purely single-role portfolio can read as narrow, especially for early-career candidates whose work has naturally spanned multiple disciplines. A purely generalist portfolio, on the other hand, reads as unfocused and fails to give recruiters a clear handle on what role to consider the candidate for. The layered approach resolves this tension by committing to a clear primary while preserving secondary and tertiary evidence that broadens the candidate's appeal. In practice, this means the hero section, skills matrix ordering, and the first three featured projects all lead with AI/ML content, while the project archive, skills matrix additional columns, and experience timeline show the broader stack.

This positioning is also resilient to role drift. If the candidate later decides to pivot toward Data Science or Full-Stack as the primary target, only the hero tagline, skills matrix ordering, and the first three featured project cards need to be re-curated. The underlying content architecture remains unchanged, which keeps the maintenance cost of pivoting low.

SECTION 04

Technology Stack

The technology stack is chosen to balance modernity, maintainability, and GitHub Pages compatibility. Every component in the stack is industry-standard as of 2026, well-documented, and supported by an active community. The candidate will be able to maintain and extend the portfolio without learning exotic tooling, and any future contributor (or AI coding assistant) will be working in a familiar environment. The table below lists each layer, the chosen technology, and the specific rationale for the choice.

Layer	Choice	Why
Framework	Next.js 16 (App Router)	Most modern React framework, statically exportable to GitHub Pages via output: export. Future-proof, with strong TypeScript support and a rich component ecosystem.
Language	TypeScript	Type safety is industry standard for serious front-end work in 2026. Catches bugs at compile time and provides excellent IDE support for portfolio maintenance.
Styling	Tailwind CSS 4	Utility-first approach enables fast iteration with consistent design tokens. Tailwind 4 brings the new Oxide engine for faster builds and better tree-shaking.
Components	shadcn/ui	Accessible, customizable component library based on Radix UI primitives. Provides a professional baseline that can be themed to match the Neon Blue x Platinum palette.
Animation	Framer Motion	The de facto standard for React animations in 2026. Enables smooth scroll reveals, magnetic hover effects, and staggered entrance animations that match the Modern Bold aesthetic.
Icons	React Icons + Simple Icons	React Icons for UI iconography, Simple Icons for tech stack logos (PyTorch, TensorFlow, React, AWS, etc.). Together they cover the full range of visual symbols a portfolio needs.
Forms	React Hook Form + Zod	Type-safe form handling with schema validation. Powers the contact form without requiring a backend, with Zod ensuring the data sent to Formspree is well-formed.
MDX	next-mdx-remote	Markdown rendering for project READMEs and any future blog content. Keeps content authoring in a portable format that survives framework changes.
Deployment	GitHub Pages	Free hosting, native GitHub integration, supports custom domain via CNAME. The username.github.io pattern gives a clean, professional URL out of the box.

4.1 Why Not Astro or Plain HTML?

Astro was seriously considered as the framework choice. It is SSG-first, ships zero JavaScript by default, and is arguably a better fit for a content-heavy static site than Next.js. The decision in favor of Next.js came down to three factors: the candidate's existing familiarity with the React ecosystem, the richer pool of open-source portfolio templates and components available for Next.js, and the option to later add dynamic features (server-side API routes, edge functions) without migrating frameworks. Plain HTML/CSS/JS was rejected because the candidate would lose the component model, TypeScript safety, and the rich Framer Motion animation ecosystem that the Modern Bold design language depends on.

SECTION 05

Design System: Neon Blue × Platinum

The design system is the single source of truth for every visual decision in the portfolio. It encodes the candidate's chosen aesthetic: a Dark Neon Blue base with Neon Silver/Platinum accents, blending the Modern Bold and Dark Sleek design languages. Every color, font, spacing unit, and motion principle below is specified concretely so that the build phase can execute without further design decisions, and so that future iterations remain visually consistent.

5.1 Color Palette (Dark Mode, Primary)

The dark mode palette is the default. It uses a deep space blue-black background to make the neon electric blue accent pop, with platinum white as the primary text color for maximum legibility. Silver tones appear as secondary highlights and icon colors, providing a cooler metallic counterpoint to the electric blue.

Token	Hex	Usage
bg-base	#050816	Page background (deep space blue-black)
bg-surface	#0a0f24	Card backgrounds
bg-elevated	#131a3a	Hover and active states
border	rgba(0, 212, 255, 0.15)	Subtle neon-tinted dividers
text-primary	#f0f4ff	Headlines and body (platinum white)
text-secondary	#a8b4d8	Captions and meta info
accent-blue	#00d4ff	Primary neon electric blue: CTAs, links, glows
accent-blue-glow	rgba(0, 212, 255, 0.5)	Box-shadows and hover halos
accent-silver	#c8d0e0	Platinum silver: secondary highlights, icons
accent-silver-glow	rgba(200, 208, 224, 0.4)	Subtle metallic glow

5.2 Color Palette (Light Mode, Toggle Option)

Light mode is provided as a toggle. It uses a soft cool white background with a deeper electric blue for contrast and a muted slate as the silver equivalent. The goal is for both modes to feel like expressions of the same brand rather than two different designs. Light mode is not the default; the dark mode is the primary expression of the portfolio's identity.

Token	Hex	Usage
bg-base (light)	#f8fafc	Cool white page background
bg-surface (light)	#ffffff	Card backgrounds
text-primary (light)	#0f172a	Deep slate headlines and body
accent-blue (light)	#0099cc	Deeper electric blue for AA contrast
accent-silver (light)	#64748b	Muted slate replaces silver

5.3 Typography

Three typefaces carry the entire portfolio. Space Grotesk handles all display and heading text: it is geometric, modern, and has a distinctly technical feel without being as cold as a pure grotesque sans. Inter handles body copy because of its class-leading legibility at small sizes on screen. JetBrains Mono appears in code snippets and terminal-style accents, reinforcing the developer identity.

Role	Family	Notes
Display / Headings	Space Grotesk	Modern, geometric, technical feel. Used at 700 weight for headings.
Body	Inter	Clean, highly readable at small sizes. Loaded at 400, 500, 600 weights.
Monospace	JetBrains Mono	Code snippets, terminal-style accents, meta labels. Loaded at 400 and 500.

5.4 Motion Principles

Motion in the portfolio is restrained but deliberate. The guiding principle is that animation should reinforce the content's hierarchy, not call attention to itself. Page load reveals the hero elements in a staggered fade-up over 0.6 seconds. Section transitions use scroll-triggered fade-ins combined with a subtle 4% scale-up to give the page a sense of depth. Hover states apply magnetic button effects, neon glow halos on cards, and subtle 3D tilt on featured project thumbnails. All motion respects the prefers-reduced-motion media query for accessibility.

- **Page load:** staggered fade-up of hero elements, 0.6s ease-out curve.
- **Section reveals:** scroll-triggered fade + 4% scale-up, triggered when 20% of the section enters viewport.
- **Hover states:** magnetic buttons (cursor-tracking translate), neon glow halos on cards, subtle 3D tilt on featured project thumbnails.
- **Smooth scroll:** native CSS scroll-behavior with anchor navigation for in-page links.

- **Accessibility:** prefers-reduced-motion respected; all non-essential animation disabled for users who request it.

SECTION 06

Site Architecture

The site architecture follows a single-page home with scroll-anchored sections, supplemented by a project archive page and individual project detail pages. This structure is the dominant pattern for modern developer portfolios: it gives recruiters the full story in a single scroll while preserving deep-linkable pages for individual projects. The resume is available both as an inline PDF preview and a one-click download, ensuring it is reachable from any page on the site.

6.1 Route Map

```
/
Home (single-page scroll, all core sections as anchored sub-routes)

/projects
Full project archive, filterable by tech stack and category

/projects/[slug]
Individual project deep-dive pages with full case study

/resume
Resume PDF download plus inline preview
```

6.2 Section Inventory

Section	Anchor	Content
Hero	#hero	Animated name reveal, tagline, one-line value prop, two CTAs (View My Work, Download Resume), animated gradient background with neon particle field, social rail (GitHub, LinkedIn, Email, optionally Kaggle/Hugging Face).
About	#about	Two to three paragraph narrative bio auto-extracted from resume and LinkedIn, quick-facts grid (location, currently learning, open to work, pronouns), optional headshot, tech philosophy pull-quote in platinum accent.
Skills	#skills	Categorized tech stack matrix: AI/ML, Data Science, Languages, Full-Stack, Tools & Infra. Each column shows floating tech logos with proficiency indicators. Final list pulled from resume and GitHub.
Experience	#experience	Vertical timeline with company, role, dates, location, two to three quantified bullets per role, tech stack chips used at each role. Sourced from LinkedIn or manually curated.
Featured Projects	#projects	Three to six curated hero project cards. Each card shows thumbnail, title, one-line hook, category badge, tech stack chips, key metric, and links to live demo, code, notebook, or paper.
Contact	#contact	Working contact form via Formspree, direct email link with mailto fallback, social rail. Optional Calendly booking link for interview scheduling.

Section	Anchor	Content
Project Archive	/projects	Auto-fetches pinned GitHub repos at build time via GitHub API, filter chips by category, search bar by tech stack or keyword, grid layout with hover-to-reveal README excerpts.

SECTION 07

AI/ML-Specific Showcase Elements

Because the portfolio's primary positioning is AI/ML, the project detail pages need to carry significantly more structured information than a typical Full-Stack project page would. Hiring managers in ML roles expect to see not just the result but the entire experimental context: the dataset, the model architecture, the training setup, the metrics, the experiment tracking evidence, the reproducibility story, and the lessons learned. The template below defines the mandatory structure for every AI/ML project detail page on the portfolio.

7.1 Mandatory Project Detail Page Structure

Field	What to Include
Problem Statement	What real-world problem does this project solve? Who is the user or stakeholder? Why does this problem matter?
Dataset	Source, size, schema, preprocessing steps. If the dataset is public, link to it. If proprietary, describe its shape and any anonymization applied.
Model Architecture	Diagram plus prose description. For deep learning, include layer-by-layer breakdown. For classical ML, include feature engineering pipeline.
Training Setup	Framework (PyTorch, TensorFlow, scikit-learn), hardware (GPU/TPU), hyperparameters, training time, number of epochs, optimization algorithm.
Metrics	Accuracy, F1, precision, recall, ROC-AUC, latency, throughput. Always include the baseline for comparison so the reader can judge the magnitude of improvement.
Experiment Tracking	Optional but high-impact: screenshots or embedded dashboards from Weights & Biases, MLflow, or TensorBoard showing the experiment progression.
Demo	Live inference demo, Colab/Kaggle notebook link, or video walkthrough. The reader should be able to interact with or see the model in action within 30 seconds.
Reproducibility	requirements.txt or environment.yml, run instructions, random seed, expected runtime on commodity hardware.
Lessons Learned	What worked, what did not, what would you do differently, and what are the next steps. This section is often what distinguishes a strong candidate from a junior one.

7.2 Data Science Project Variant

Data Science projects (the secondary positioning) follow a slightly different template that emphasizes the analytical narrative over the model. The structure is: business problem, data acquisition and cleaning, exploratory data analysis with at least three visualizations, methodology (statistical or ML), results with

business-impact framing, and recommendations. The visualizations are embedded inline using Plotly or static PNG exports from Matplotlib, with the underlying notebook linked from a "View Notebook" button.

7.3 Full-Stack Project Variant

Full-Stack projects (the tertiary positioning) follow a structure that emphasizes system design and deployment: problem statement, system architecture diagram, tech stack table, frontend highlights (UI screenshots, component design), backend highlights (API design, database schema, performance optimizations), deployment and CI/CD pipeline, and a live demo link. The goal is to demonstrate that the candidate can ship a complete product, not just write model code.

SECTION 08

Features Checklist

The features below are the explicit set the candidate selected during the design clarification round. Each feature is mapped to its implementation approach so that the build phase has a concrete plan for every item. Features are grouped into functional, accessibility, performance, and SEO categories for clarity.

8.1 Functional Features

Feature	Implementation
Dark / Light Toggle	next-themes provider wrapping the app, default to dark. Toggle button in nav with system-preference fallback. LocalStorage persistence.
Contact Form	React Hook Form + Zod validation. Submission via Formspree endpoint (no backend required). Success/error states with accessible ARIA announcements.
Resume PDF Download	Versioned PDF in /public/resume/. Download button in nav, hero, and footer. Optional inline preview via react-pdf.
Project Archive Filtering	Client-side filtering by category and tech stack. Built at static export time from a JSON manifest; auto-pulls pinned GitHub repos via the GitHub REST API at build time.
Featured Project Cards	Curated list of 3-6 hero projects, each with its own MDX-driven detail page. Category badges, tech stack chips, and a key metric field are mandatory on every card.

8.2 SEO and Metadata

Feature	Implementation
Per-Page Meta Tags	Next.js Metadata API. Title, description, OG image, Twitter card per route. Default values in root layout, overridden per page.
Open Graph Images	Auto-generated via @vercel/og at build time. Each project page gets a custom OG card with the project title and metric.
Sitemap	Auto-generated sitemap.xml via Next.js built-in sitemap() function. Includes all routes and project detail pages.
Robots.txt	Static robots.txt in /public/. Allows all crawlers, points to sitemap.
JSON-LD Person Schema	Structured data in the root layout describing the candidate: name, jobTitle, sameAs (GitHub, LinkedIn), knowsAbout (skills).

8.3 Accessibility and Performance

Feature	Implementation
WCAG AA Compliance	Semantic HTML, ARIA labels on interactive elements, keyboard navigation for all features, color contrast verified at AA level for both dark and light modes.
Reduced Motion Support	All non-essential animations wrapped in a prefers-reduced-motion check. Framer Motion useReducedMotion hook applied at the layout level.
Mobile-First Responsive	Tailwind breakpoints tested at 360px, 768px, 1024px, 1440px. Mobile nav uses a slide-in drawer, all touch targets minimum 44x44px.
Lighthouse 90+	Static generation for fast first paint, lazy-loaded images with next/image, font-display: swap on all web fonts, no render-blocking third-party scripts.
Image Optimization	next/image with WebP/AVIF automatic format negotiation. Responsive srcset for thumbnails. Blur placeholder for above-the-fold images.

SECTION 09

Deployment Plan

Deployment to GitHub Pages follows a well-trodden path. The build is a standard Next.js static export, the repository naming convention gives a free root-level subdomain, and the GitHub Actions workflow handles automatic redeployment on every push to main. The entire pipeline is reproducible from a fresh clone in under five minutes, and the custom domain migration path (should the candidate later purchase a domain) requires zero code changes.

9.1 Step-by-Step Deployment

- **Build locally** with `next build` to produce the static export in the `out/` directory. Verify the export completes without errors and that all routes render correctly when served locally.
- **Create the GitHub repository** named `.github.io`. This exact naming convention is required for the free root-level subdomain; any other name will result in a `username.github.io/repo-name/` URL structure.
- **Push the contents** of the `out/` directory to the main branch of the new repository. Alternatively, push the source code and use a GitHub Action to build and deploy automatically on every push.
- **Enable GitHub Pages** in the repository Settings → Pages → Source: main branch. The site will be live at `https://.github.io/` within approximately two minutes.
- **Verify the deployment** by visiting the URL in an incognito window, running a Lighthouse audit, and testing all interactive features (theme toggle, contact form, project filters, navigation).
- **(Future) Custom domain:** if the candidate later purchases a custom domain, add a `CNAME` file to the repository root containing the domain name, and configure DNS at the registrar. No code changes are required.

9.2 GitHub Action (Optional but Recommended)

A GitHub Action that builds and deploys on every push to main is recommended over manual deployments. The workflow uses the official Next.js build action, exports the static output, and pushes it to a gh-pages branch that GitHub Pages serves. This pattern keeps the source code in main and the deployed artifact in gh-pages, which is cleaner than committing the build output directly to main. The action takes approximately 90 seconds to run end-to-end on a typical portfolio repository.

SECTION 10

Build Phases (Execution Order)

The build is divided into six phases, each with a clear deliverable. Phases are sequential: each phase depends on the artifacts of the previous one, and jumping ahead will produce rework. The phase boundaries are also the natural review checkpoints: at the end of each phase, the candidate can inspect the deliverable and request adjustments before the next phase begins.

Phase	What Ships	Effort
P1: Scaffold + Design System	Next.js 16 project initialized, Tailwind configuration, design tokens (colors, typography, spacing) encoded as CSS variables, theme provider wired up, base layout with nav and footer. No real content yet.	Foundation
P2: Core Sections	Hero, About, Skills, Experience sections built with placeholder content. All animations and scroll-reveal behavior wired up. Mobile responsive at this stage.	Content Layer
P3: Projects System	Featured Projects section, project archive page, individual project detail pages, GitHub API integration for pinned repo auto-pull. Project MDX content for 3-6 featured projects.	Showcase Layer
P4: Contact + Resume + Extras	Contact form (Formspree integration), resume download, social links, footer polish. Theme toggle, analytics, and SEO metadata wired up.	Conversion Layer
P5: Polish + SEO + Performance	Per-page meta tags, OG card generation, sitemap.xml, robots.txt, JSON-LD Person schema. Lighthouse audit pass. Accessibility audit. Mobile testing at four breakpoints.	Quality Layer
P6: Deploy to GitHub Pages	Static export configuration, GitHub Action for auto-deploy, go-live checklist, final verification on the live URL.	Launch

10.1 Review Checkpoints

At the end of each phase, the candidate should review the deliverable and either approve progression to the next phase or request changes. The phases are designed so that changes at any phase do not invalidate the work of previous phases. For example, requesting a different hero animation in P2 does not require re-doing the design system work in P1. This keeps the iteration loop tight and predictable.

SECTION 11

Inputs Required Before Build

Before the build can begin, the candidate needs to provide the materials listed below. The required items are the master content source and the two profile URLs that drive automated content pulling. The optional items are high-impact additions that materially improve the portfolio but can be added later if not immediately available.

11.1 Required Materials

Item	Format	Why Needed
Resume	PDF or pasted text	Master content source. Drives the About, Experience, Skills, and Education sections. The PDF is also embedded as the downloadable resume.
LinkedIn Profile URL	URL	Source for experience entries, education history, certifications, and recommendations. Cross-checks and supplements the resume.
GitHub Profile URL	URL	Drives the auto-pull of pinned repositories for the project archive. Also populates the GitHub activity panel and contribution graph if that section is enabled.

11.2 High-Impact Optional Materials

Item	Why It Helps
Target job titles and dream companies	Lets the build tune the hero messaging, project ordering, and skills emphasis to match the specific roles and companies being targeted.
Project screenshots and demo URLs	For the 3-6 featured projects. If not provided, the build will pull from the project READMEs on GitHub, which is acceptable but less polished.
Headshot photo	Strictly optional. Many developer portfolios skip this in 2026, but some candidates prefer to include one in the About section. The build supports either choice.
Email address	Required for the contact form (Formspree requires a verified email) and the mailto link in the contact section.
Analytics preference	Plausible (privacy-friendly, free self-host or $9/month cloud) versus Google Analytics 4 (free, more data, less privacy-focused). The build will integrate whichever is chosen.

11.3 Verification Request

Before any code is written, the candidate is asked to verify this plan. Specifically: does this plan match the candidate's vision, are there any sections to add, remove, or reorder, are there specific projects the candidate knows they want featured, and is the candidate ready to share the resume, LinkedIn, and GitHub materials. Once verification is received and the materials are shared, the build will proceed through the six phases end-to-end and deliver a deployed portfolio at the candidate's `username.github.io` URL.